ORE Studio User Manual

Marco Craveiro

Version 0.0.18

Contents

Introduction

About ORE Studio

ORE Studio is a desktop application for exploring, configuring, and running quantitative finance calculations. It provides a graphical environment built around the *Open Source Risk Engine* (ORE), a widely-used open-source library for pricing derivatives and measuring financial risk, which is itself built on QuantLib. ORE Studio handles the data — storing trades, market data, and model configurations, and presenting the results — while ORE and QuantLib provide the mathematics.

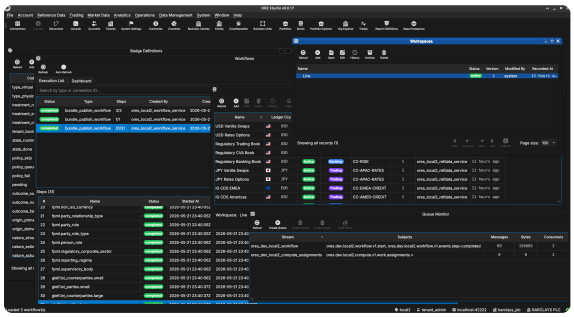

Figure 1: ORE Studio v0.0.17 — the main workspace showing the instrument and trade management views.

Audience

This manual is written for several kinds of reader:

- **Analysts, traders, quants, and middle-office staff** who want to explore financial instruments and risk calculations through a graphical interface. No programming experience is required.
- **Students and researchers** learning quantitative finance. ORE Studio makes it possible to experiment with derivative pricing, yield curve construction, and risk measures without writing code.
- **LLM-based agents and coding assistants** that interact with ORE Studio on a user's behalf, help configure workspaces, interpret results, or draft analytical reports. Multimodal models — those capable of processing both text and images — are preferred, as the manual makes extensive use of screenshots to describe the user interface.

The focus throughout is on what you can do through the application itself: entering trades, configuring market data, running analytics, and understanding the results.

Functional Areas

ORE Studio is organised around a set of functional areas accessible from the main menu:

- *Reference data* — currencies, business calendars, counterparties, and market conventions that form the foundation for everything else.
- *Instruments and trades* — define, store, and manage financial instruments and their terms.
- *Market data* — yield curves, volatility surfaces, fixing histories, and other inputs to pricing and risk models.
- *Model configuration* — choose and parameterise pricing engines, simulation models, and sensitivity specifications.
- *Analytics* — run calculations and view results: net present value, sensitivities (Greeks), credit and funding valuation adjustments (XVA), Monte Carlo simulations, and stress tests.

All data is stored persistently, so trades, curves, and configurations can be saved, revisited, and reused across sessions.

What ORE Studio Is Not

ORE Studio is a learning and exploration environment, not a production trading or risk system. It is independent of and unaffiliated with ORE, QuantLib, or any financial institution. All quantitative mathematics are provided by ORE and QuantLib; ORE Studio is the surface through which they are configured and their results explored. Users who need production-grade performance, real-time market data feeds, or regulatory reporting should look to enterprise risk platforms.

How This Manual Is Organised

The chapters follow the natural order of first use. After this introduction, *Connecting to ORE Studio* covers launching the application, establishing a connection to the backend, and orienting yourself in the main window; *Initial Setup* then covers provisioning the system, tenants, and parties. Later chapters cover each functional area in turn. You do not need to read the manual cover to cover; each chapter can be read independently once the system is up and running.

Early-Stage Software Notice

Early-stage software. ORE Studio is currently in active development (version 0.x). Features, workflows, and this documentation are all evolving and may change between releases. Numbers produced by the system are for learning and exploration only — they must not be used for real trading, risk management, or any financial decision-making.

A Note on This Document

This manual was generated entirely by large language models (LLMs) working under human supervision. While every effort has been made to ensure accuracy, LLM-generated content can contain errors, omissions, or descriptions that do not match the actual software behaviour. If you find a discrepancy between this manual and the application, trust the application. Please report inaccuracies via the project issue tracker so they can be corrected.

Connecting to ORE Studio

Getting started

When you launch ORE Studio a splash screen appears briefly while the application initialises, showing the ORE Studio logo and the build version in its lower-right corner.

Figure 2: The ORE Studio splash screen shown during start-up. The build version appears in the lower-right corner.

Once initialisation completes you are greeted by the main window in its *disconnected* state. The application is running but has not yet established a connection to the backend service that performs calculations. Before you can work with trades, market data, or analytics, you need to connect to a running ORE Studio backend.

This chapter covers:

- What the main window looks like before a connection is established.
- How to open the Connection Manager and configure one or more backends.
- How to log in once a connection is active.
- How to manage your saved connections — adding, editing, deleting, organising by environment and tag.
- How to read the status bar and understand what each zone shows.
- How to launch ORE Studio from the command line with options for telling windows apart, configuration directory, auto-connect, and diagnostics.

The main window before login

In the disconnected state most of the application is inactive. The menu bar and toolbar are present but workspace panels, trade lists, and analytics views are all unavailable until a successful login. Two toolbar buttons are always accessible:

Figure 3: The ORE Studio main window immediately after launch, in the disconnected state. The toolbar shows the Connect and Connections buttons; the workspace area is empty until you log in.

- **Connect** — opens the login dialog for the currently selected connection.
- **Connections** — opens the Connection Manager, where you configure the list of backends ORE Studio knows about.

The Connection Manager

The Connection Manager is the central place where you maintain the list of ORE Studio backends the application can connect to. You might have a local development backend, a shared team backend, and a staging backend — each appears as a separate entry here. The manager lets you add, edit, delete, and organise those entries before you attempt a login.

Open it at any time from the toolbar **Connections** button or from the *File* menu.

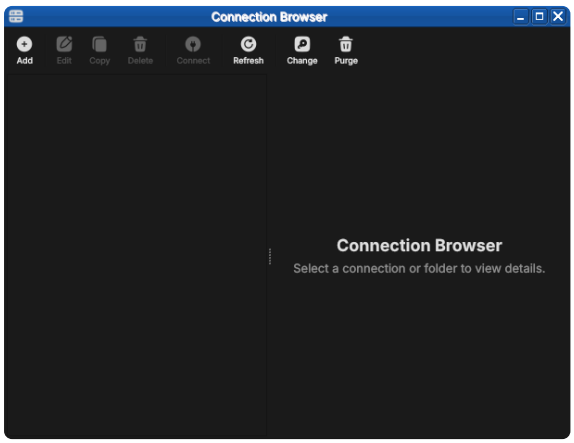

Figure 4: The Connection Browser open with no connections configured. The toolbar exposes Add, Edit, Copy, Delete, Connect, Refresh, Change and Purge; the unusable actions are greyed out until something is selected.

Adding a new connection

Click **Add** (or the "+" button) to open the *New Connection* form. Fill in the fields:

- **Name** — a free-text label you choose, used throughout the UI to identify this backend (e.g. "Local dev", "Team staging"). Must be unique among your saved connections.

- **Host** — the hostname or IP address of the machine running the ORE Studio backend (e.g. `localhost`, `192.168.1.10`, `ores-staging.example.com`).
- **Port** — the TCP port the backend is listening on. The default is `35900`; change it only if the backend was started on a different port.
- **Environment** — an optional grouping label (see Environments below). Helps you distinguish development, staging, and production backends at a glance.
- **Tags** — zero or more free-text labels (see Tags below). Useful for filtering and searching when you have many connections.

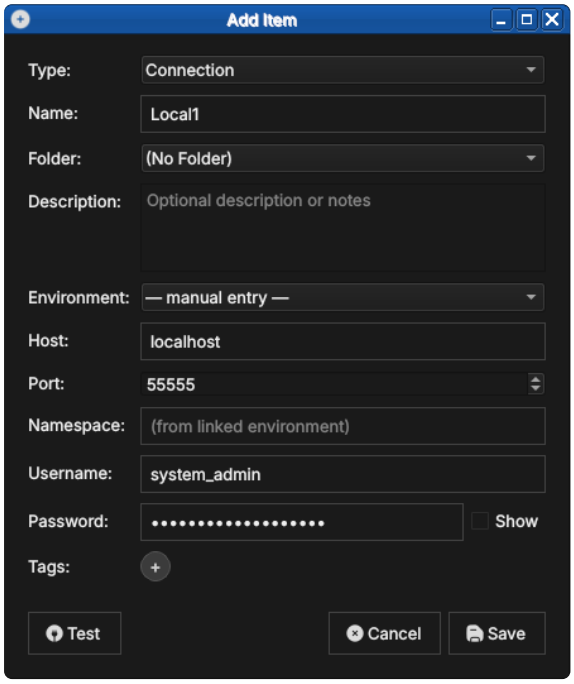

Figure 5: The New Connection form with example values filled in. *Type* is set to `Connection`; the *Environment* may be left as *manual entry* or linked to a defined environment.

Click **Save** (or **OK**) to add the connection to the list. The new entry appears immediately in the Connection Manager list. No network contact is made at this point — ORE Studio simply stores the configuration.

Editing a connection

Select an existing connection in the list and click **Edit** (or double-click the row) to open the same form pre-populated with the saved values. Change any field and click **Save**. The connection list updates immediately.

Deleting a connection

Select one or more connections in the list and click **Delete** (or press the Delete key). A confirmation dialog asks you to confirm the removal.

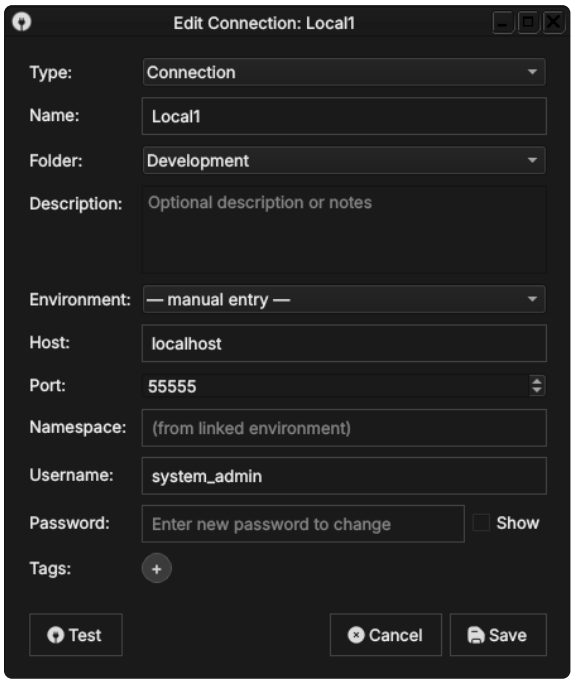

Figure 6: Editing a saved connection. The dialog title shows *Edit Connection*, the fields are pre-populated, and the password field reads *Enter new password to change* — the stored password is preserved unless you type a replacement.

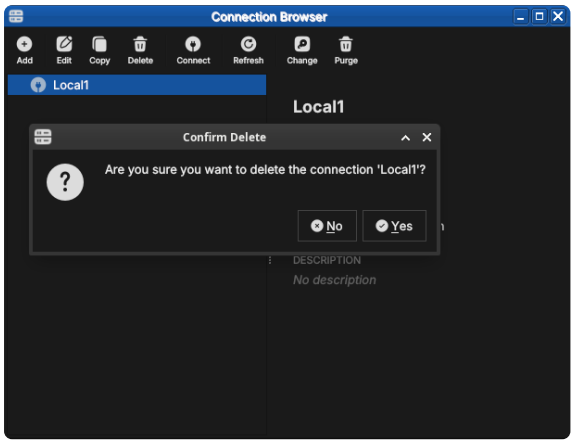

Figure 7: The delete confirmation dialog. ORE Studio names the connection being removed and waits for you to confirm with *Yes* or cancel with *No*.

Deleting a connection removes only the saved configuration; it has no effect on the running backend or any data stored there.

Organising connections into folders

When the list grows, *folders* keep it manageable. A folder is a named container you can nest, grouping related connections — by team, by client, or by environment tier. In the populated browser above, the connections sit under a `Development → Dev → Local1...Local4` folder tree.

To create one, click **Add** and set *Type* to `Folder`. Give it a name and an optional description, and choose a parent folder (or *No Folder* to place it at the top level). Connections are then assigned to a folder through their *Folder* field, or by dragging them in the tree.

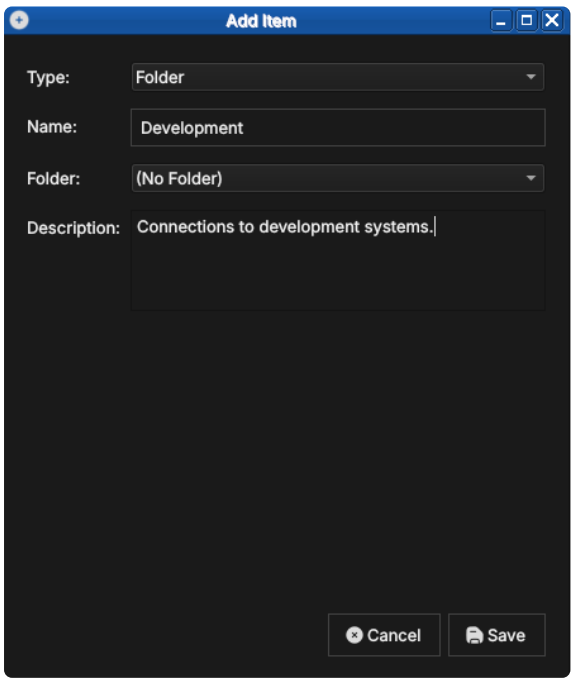

Figure 8: Creating a folder. With *Type* set to `Folder`, a folder needs only a name and an optional description; it can be nested inside another folder.

Protecting your credentials with a master password

The passwords you save with a connection are encrypted at rest using a *master password* that you choose. The first time you save a credential, ORE Studio offers to create one.

Figure 9: Creating the master password. It encrypts every saved server password; there is no recovery if you forget it, so you may also leave it blank to store passwords unencrypted.

Type the same value into *New Password* and *Confirm Password*; the

fields turn green when they match. *Show password* reveals what you typed.

Figure 10: The master password confirmed — both fields match and are outlined in green, ready to accept with *OK*.

When you next launch ORE Studio and open the Connection Browser, it prompts you to unlock your saved connections by entering the master password. Until you do, the encrypted passwords stay sealed.

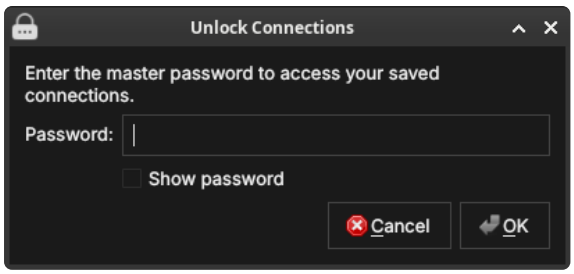

Figure 11: The unlock prompt shown on a later launch. Enter the master password to decrypt your saved connection credentials.

If you leave the master password blank, your connections still work but their passwords are stored unencrypted — acceptable on a personal machine, but not recommended on shared or portable systems.

Environments

An *environment* is a named grouping that you assign to a connection to indicate which tier of your infrastructure it belongs to. Typical values are `Development`, `Staging`, and `Production`, but the list is fully customisable — you can define any environment names that make sense for your setup.

Environments appear as a coloured badge or label next to the connection name in the list, making it immediately obvious which tier you are about to connect to. This is a safety feature: it is easy to accidentally connect to production when you meant staging; a clearly labelled environment badge prevents that.

To manage the available environment names, open the *Environments* section within the Connection Manager settings (or the dedicated menu item). From there you can:

- **Add** a new environment name and choose its display colour.
- **Rename** an existing environment (all connections using it update automatically).
- **Delete** an environment (connections that used it revert to *Unassigned*).

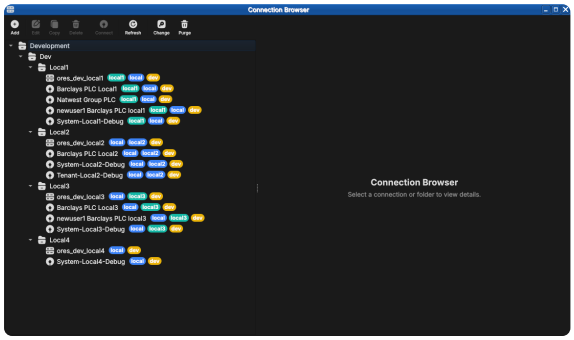

Figure 12: A populated Connection Browser. Connections are grouped under folders (here `Development → Dev → Local1...Local4`) and each carries coloured chips — the environment and tags — making the tier and purpose of every entry obvious at a glance.

An environment is itself created from the Connection Browser: click **Add** and set *Type* to `Environment`. An environment carries its own host, port, HTTP port, namespace and tags — these become the connection details that any connection linked to it inherits.

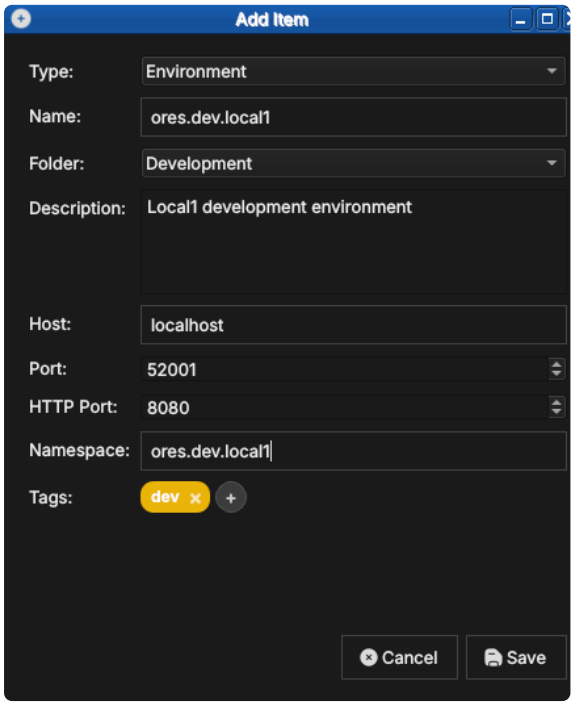

Figure 13: Creating an environment. With *Type* set to `Environment`, you give it a name, folder, host, ports, namespace, and tags (here a `dev` tag).

Once an environment exists, a connection can link to it by selecting it in the connection's *Environment* field instead of *manual entry*. The connection then inherits the environment's host, port, and namespace, so you maintain those details in one place.

Tags

Tags are free-text labels you attach to a connection to enable flexible filtering and search. Unlike environments (which represent a single tier), a connection can carry any number of tags. Examples: `personal`, `client-acme`, `read-only`, `high-memory`.

Tags are displayed inline with the connection name in the list. The Connection Manager provides a tag filter bar at the top of the list: type or select a tag to show only the connections that carry it.

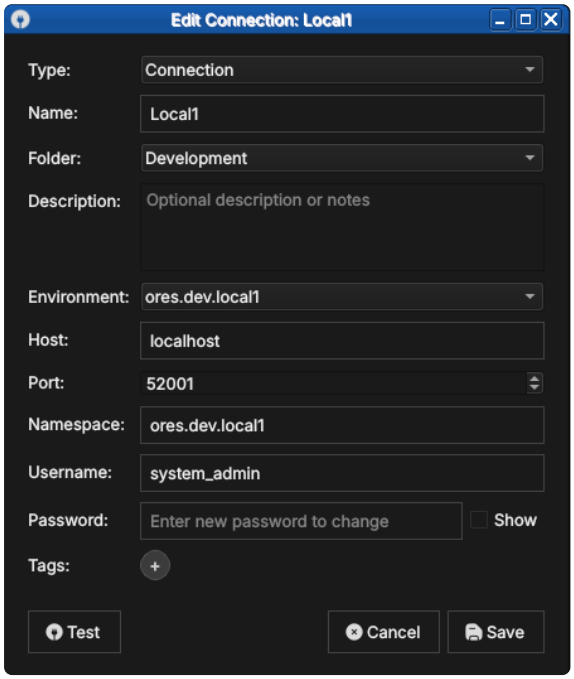

Figure 14: A connection linked to an environment. Selecting `ores.dev.local1` in the *Environment* field populates the host, port and namespace from that environment.

To add or remove tags, open the Edit form for the entry and use the *Tags* field. Type a new tag name and press Enter (or comma) to add it; click the × on an existing tag chip to remove it. Tags are created on first use — there is no separate tag management screen.

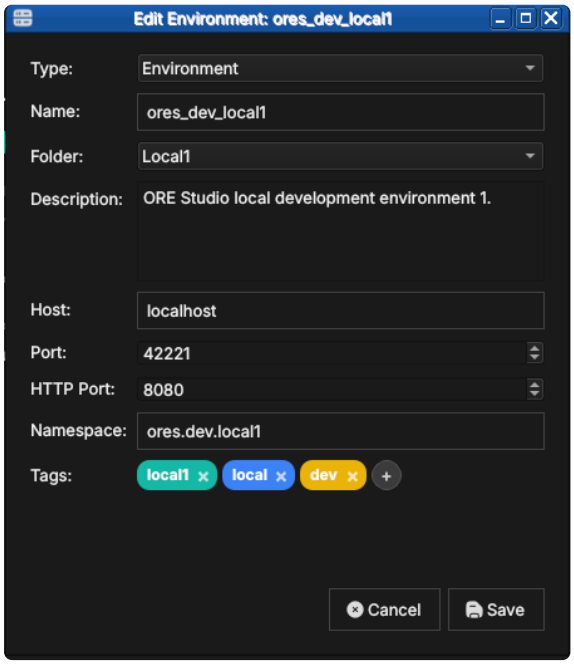

Figure 15: Tags shown as coloured chips in an entry's Edit form (here the `ores_dev_local1` environment, tagged `local1`, `local` and `dev`). Add tags in the *Tags* field, remove one with the × on its chip, or click + to add another.

Where your connections are stored

ORE Studio keeps your connections — together with their environments, folders, and tags — in a single local SQLite database file named `connections.db`. Your UI settings (preferences and window

layout) are stored separately, in the operating system's standard settings store. Neither leaves your machine.

- **Linux**:
  - settings in `~/.config/OreStudio/`;
  - database at `~/.local/share/ores.qt/connections.db`.
- **macOS**:
  - settings in `~/Library/Preferences/`;
  - database at `~/Library/Application Support/ores.qt/connections.db`.
- **Windows**:
  - settings in registry: `HKCU\Software\OreStudio\OreStudio`;
  - database at `%APPDATA%\ores.qt\connections.db`.

If `connections.db` is missing when ORE Studio starts, it creates a new empty one — so the Connection Manager simply opens with no entries.

Backing up and restoring your connections

Because everything you configure here lives in that one `connections.db` file, backing it up is a file copy:

1. Quit ORE Studio, so the database is fully written and not locked.
2. Copy `connections.db` from the location above to a safe place — on Linux, for example:

```
cp ~/.local/share/ores.qt/connections.db ~/connections-backup.db
```

To restore, quit ORE Studio and copy the backup back over `connections.db`. To move your connections to another machine, copy the file to the same location there. The file is self-contained, so a single copy preserves all your connections, environments, folders, and tags.

Connecting and logging in

Once you have at least one connection configured, you can log in.

Selecting the active connection

In the Connection Manager, select the connection you want to use and click **Set as active** (or double-click it). The selected connection becomes the target for the **Connect** button in the main toolbar. The status bar at the bottom of the main window shows the name of the active connection.

You do not have to choose in advance, though: clicking **Connect** opens the login dialog directly, and its *Quick Connect* selector lets you pick the connection there (see Filtering Quick Connect with labels below).

The login screen

With an active connection selected, click **Connect**. ORE Studio attempts to reach the backend. If the backend is reachable, the *Login* dialog appears.

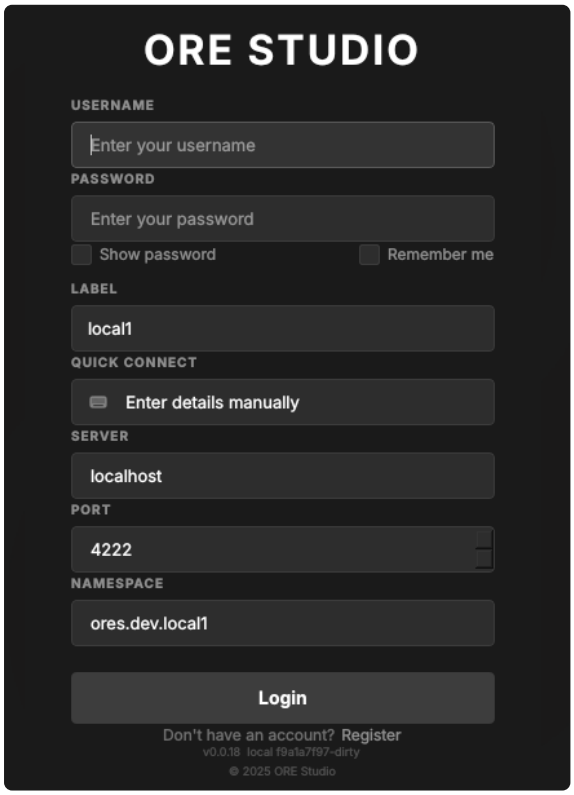

Figure 16: The login dialog. Below the credentials it shows the connection *Label*, a *Quick Connect* selector, and the *Server*, *Port* and *Namespace* the login will target; the footer carries the *Register* link and the build version.

Enter your **username** and **password** and click **Login**. Tick *Remember me* to have ORE Studio recall the username next time. ORE Studio authenticates against the backend's user database. On success the dialog closes and the main window transitions to the *connected* state: the workspace panels become active, the menu bar is fully enabled, and the status bar shows your username and the connected backend name.

The fields below the credentials describe *which* backend you are logging in to:

- **Label** — the saved connection's name (e.g. `local1`).
- **Quick Connect** — pick a saved connection to fill *Server*, *Port* and *Namespace* in one step, or leave it on *Enter details manually* to type them yourself.
- **Server**, **Port**, **Namespace** — the backend address the login targets.

The *Label* and *Quick Connect* fields work together. The *Label* names the environment — it defaults from the connection (or from the `--instance-name` you launched with); *Quick Connect* then offers the connections for that label, and picking one fills in *Server*, *Port* and *Namespace* so you do not have to type them.

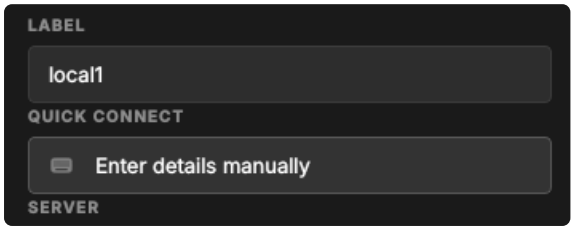

Figure 17: The *Label* and *Quick Connect* fields. The Label selects the environment; Quick Connect offers that environment's connections and fills in the server details.

If you do not yet have an account, the **Register** link in the footer opens account registration. If authentication fails, an error message is shown below the password field. Check that you are using the correct credentials for this backend; each backend maintains its own user database independently.

Filtering Quick Connect with labels

With only a handful of connections, *Quick Connect* is easy to scan. But a real deployment soon accumulates many — several environments, each with its own system, tenant and per-party logins. Left unfiltered (the *Label* set to *All*), the selector lists every one:

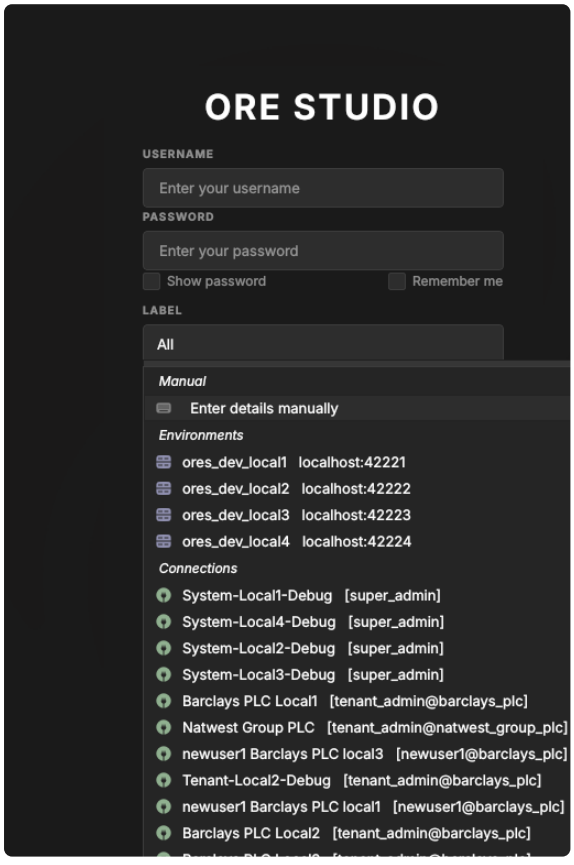

Figure 18: Quick Connect with the *Label* set to *All* — every saved connection across all environments and tenants. Hard to pick the right one at a glance.

Each connection carries a *label* — its environment tag, such as `dev`, `local1` or `local2`. The *Label* selector lists those labels; choosing one filters Quick Connect to just the connections that carry it:

With `local1` selected, Quick Connect shows only the `local1` entries — the environment's own database, its system and tenant lo-

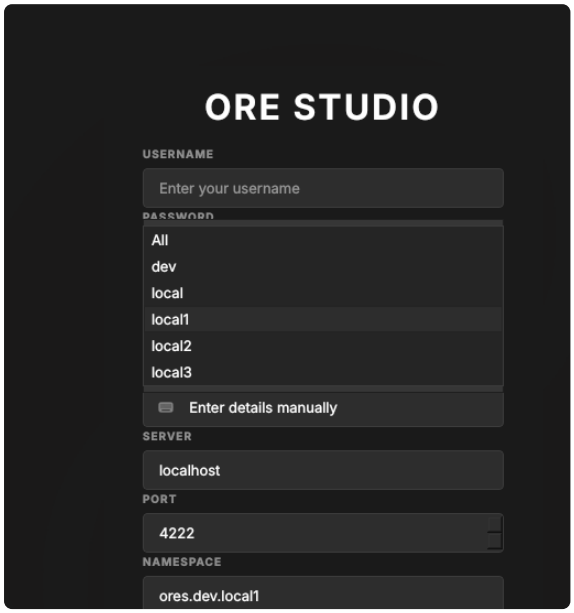

Figure 19: The *Label* selector lists the labels found on your connections. Pick one to narrow Quick Connect to that environment.

gins, and its per-party connections — so you reach the right backend without scrolling past unrelated ones:

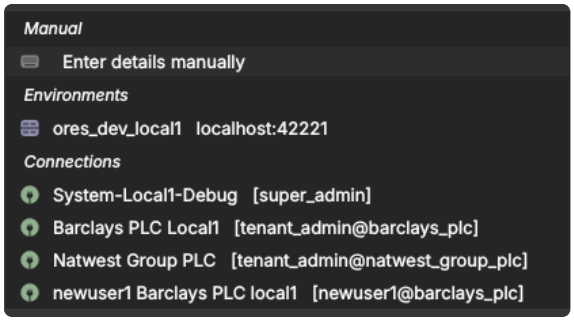

Figure 20: Quick Connect filtered to the `local1` label: only that environment's connections remain.

You normally run one client *per environment*: a client built for `local1` should connect to `local1`, not to staging or production. Rather than pick the label by hand each time, set it when you launch the client — the `--instance-name` command-line option (see Telling windows apart) opens the client with the *Label* already set, so Quick Connect is pre-filtered to that environment. The `build/scripts/start-client.sh` helper does this for you, taking the label from `ORES_CHECKOUT_LABEL` in your `.env`.

When the connection fails

Not every login failure is a wrong password. ORE Studio talks to the backend over a messaging server (NATS), and if it cannot reach that server — or the services behind it — it tells you which is wrong rather than just reporting "login failed".

If the messaging server itself is unreachable, ORE Studio reports that it cannot connect, along with the host and port it tried:

If the messaging server is running but the application services behind it have not started, ORE Studio says so explicitly — the fix is

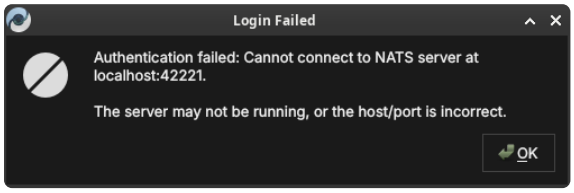

Figure 21: A login failure caused by an unreachable messaging server. The host and port ORE Studio tried are shown; the server is likely not running, or the host/port is wrong.

to start the backend services, not to change your connection:

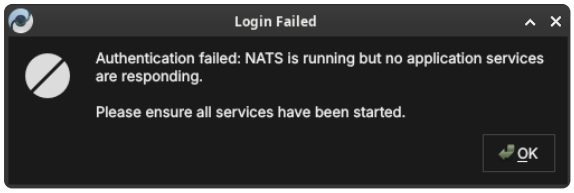

Figure 22: A login failure where the messaging server is up but no application services are responding. Ensure the backend services have been started.

In both cases your saved connection is fine; correct the backend (start the server or its services) and click **Connect** again.

First login and the default account

On a freshly initialised backend the only account that exists is the default administrator account. Your system administrator will have provided the initial credentials. After first login it is strongly recommended to change the password immediately via *Settings → Account*.

Once a tenant has been provisioned (covered in the next chapter, *Initial Setup*), you log in with that tenant's administrator account — the username takes the form `user@tenant_code`. Picking the connection from *Quick Connect* fills in the server, port and namespace for you:

Connection status and reconnection

The status bar always shows whether you are connected. After a deliberate log-out — or once a stopped backend is running again — use the **Connect** button to re-establish the session: your saved connection is retained, so you only re-enter your password.

Note: ORE Studio does not yet reliably detect a backend that drops *while you are logged in*. If the messaging server (NATS) or the services behind it are stopped mid-session, the client may keep looking connected until your next action fails or hangs. Detecting the drop and showing a clear connection-lost state is planned work, tracked in the story *Detect and report NATS disconnection*.

The status bar

The status bar runs along the bottom of the main window and provides a continuous read on the application's connection state. It is always visible regardless of what is open in the workspace.

The status bar is divided into zones from left to right:

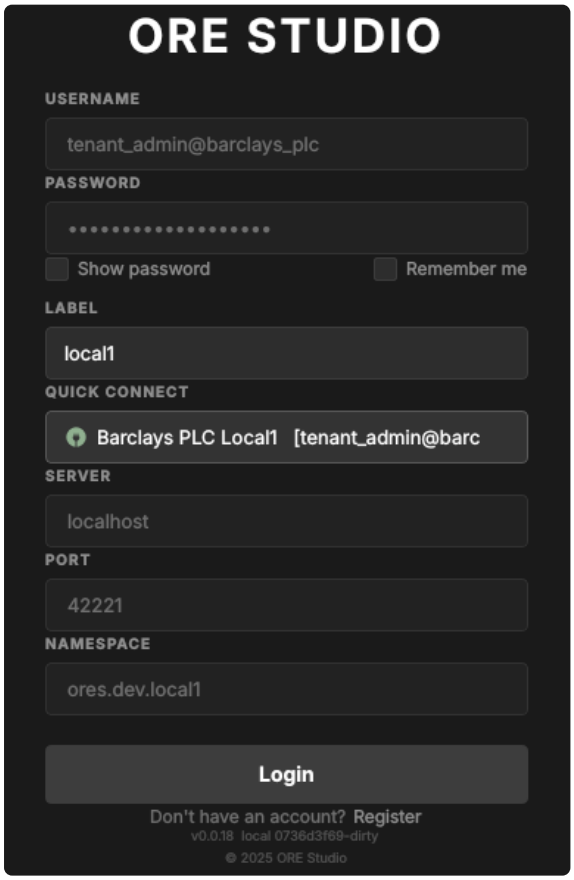

Figure 23: Logging in as a tenant administrator (`tenant_admin@barclays_plc`), with the connection chosen from *Quick Connect*.

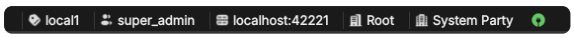

Figure 24: The status bar in the fully connected, logged-in state: from left to right, the environment marker (`local1`), the username (`super_admin`), the server (`localhost:42221`), the active party (Root / System Party), and the green connected indicator.

- **Connection name** — the name of the active connection as entered in the Connection Manager (e.g. "Local dev"). Clicking this zone opens the Connection Manager directly.

- **Environment badge** — the coloured environment label (e.g. `Production` in red) if one is assigned to the active connection. Absent when no environment is set. This is the most prominent safety indicator: always glance at the environment badge before performing any write operation.

- **Username** — the account name of the currently logged-in user. Absent in the disconnected state.

- **Server** — the backend address (host and port) the session is connected to, e.g. `localhost:42221`.

- **Active party** — the party context for the session (e.g. `Root / System Party`), set at login when your account spans multiple parties.

- **Connection indicator** — a coloured icon at the right showing the live link state (green when connected).

The status bar changes appearance to reflect the connection state:

- **Disconnected** (no active connection) — grey; shows "Not connected".

- **Connecting** (handshake in progress) — animated indicator; shows "Connecting to /name/...".

- **Connected and logged in** — normal; shows all zones as described above.

(A distinct *connection lost* state for a backend that drops mid-session is planned but not yet implemented — see Connection status and reconnection above.)

In the disconnected state the strip is reduced to the environment marker and the connection label, with the broken-link icon on the right:

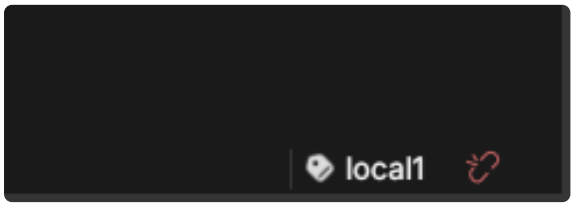

Figure 25: The status bar in the disconnected state, showing the environment marker, the connection label (`local1`), and the broken-link indicator.

The connected state is shown at the start of this section. Compare it with the disconnected strip above to recognise at a glance which state the application is in.

Starting ORE Studio from the command line

ORE Studio can be launched directly from a terminal, which is useful for scripting, for telling several windows apart, or for pointing the application at a non-default configuration directory. Run `ores --help` to see the full list of accepted options for your installed version; the most commonly used ones are documented below.

```
ores --help
```

Telling windows apart (instance colour and name)

When you run more than one ORE Studio window at once — for example against different backends, or from separate checkouts — you can give each window a distinct identity so you can tell them apart at a glance. This is *not* a light/dark theme; it is purely a per-window marker.

- `--instance-color HEX` draws a small coloured circle in the status bar in that colour (a 6-digit RGB hex, e.g. `F44336` for red). Give each window a different colour.
- `--instance-name NAME` (short form `-n`) labels the window with a name, also shown in the status bar, so you can see which window is which.

```
ores --instance-name "Local dev" --instance-color 2196F3
```

The colour and the name are independent: the colour is only a visual marker, and the name is what identifies the instance. If you launch via `build/scripts/start-client.sh`, its `--colour red|green|blue|<hex>` sets the marker colour and `--name` sets the instance name; when you omit `--name`, the name comes from `ORES_CHECKOUT_LABEL` in your `.env` — never from the colour.

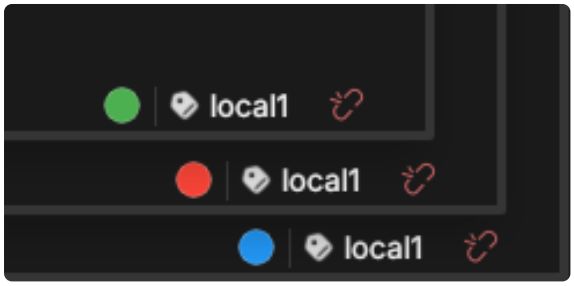

Figure 26: The status bars of three windows running the same `local1` instance, each given a different `--instance-color` (green, red, blue) so you can tell them apart at a glance.

Configuration directory

By default ORE Studio stores its UI settings — preferences and window layout — in the operating system's standard settings store. The exact per-OS locations are listed under Where your connections are stored; your saved connections live separately in `connections.db`. To use a different directory:

```
ores --config-dir /path/to/config
```

This is useful when running multiple isolated instances, or when keeping configuration under version control for team-shared settings.

Selecting a connection at startup

To bypass the Connection Manager and connect immediately to a named connection:

```
ores --connection "Local dev"
```

ORE Studio will start, set the named connection as active, and open the login dialog automatically. The connection name must match exactly (case-sensitive) a saved entry in the Connection Manager.

Logging and diagnostics

For troubleshooting, verbosity can be increased:

```
ores --log-level debug    # verbose output to stdout
ores --log-file /tmp/ores.log  # write log to a file instead
```

Log output includes connection lifecycle events, authentication attempts, and backend protocol messages. The `debug` level is intended for issue reporting and development; it produces high-volume output and should not be left enabled during normal use.

Finding the application version

Knowing exactly which build you are running matters when reporting an issue or checking client/backend compatibility. ORE Studio shows its version in several places.

The title bar of the main window always shows the version next to the application name:

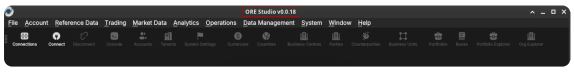

Figure 27: The version in the main window title bar, alongside the full menu bar and toolbar of the connected application.

It also appears in the lower-right corner of the splash screen at start-up:

Figure 28: The build version in the lower-right corner of the splash screen.

. . . and in the footer of the login dialog, below the *Register* link:

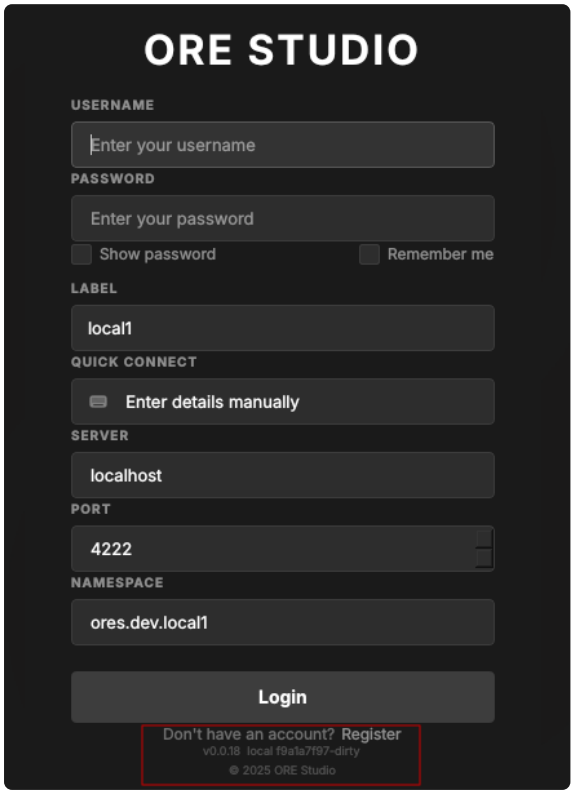

Figure 29: The version string in the login dialog footer, beneath the *Register* link.

From the command line, `ores --version` prints the client version string (e.g. `ORE Studio 0.0.19`) and exits — handy in scripts or when filing a bug report.

```
ores --version
```

Running in the background and the system tray

While ORE Studio is running it places an icon in your desktop's system tray (notification area). The icon keeps the application reachable when its window is minimised or hidden, and shows the instance name so you can tell multiple windows apart.

Figure 30: The ORE Studio icon in the system tray, labelled with the instance name so you can identify the window it belongs to.

Logging out

To end your session, choose *File* → *Log Out* or click the **Disconnect** toolbar button. ORE Studio closes the connection to the backend and returns to the disconnected main window. Your saved connection configurations are preserved; you can log back in at any time.

Logging out does not stop the backend service — it only terminates your client session. Other users connected to the same backend are unaffected.

To close the application entirely, quit the window (or choose *File → Exit*). ORE Studio asks you to confirm so you do not lose an active session by accident:

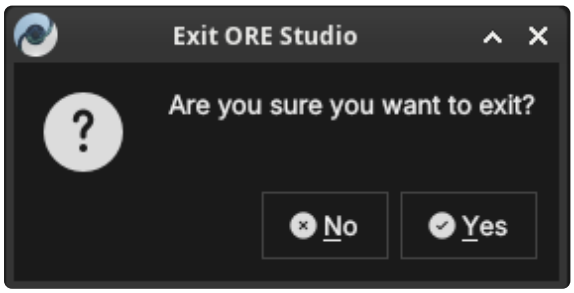

Figure 31: The exit confirmation. Click *Yes* to close ORE Studio, or *No* to keep working.

Initial Setup

Understanding the multi-tenant, multi-party model

Before you can use ORE Studio for the first time, the system must be bootstrapped — meaning the initial administrative structure must be established. Understanding why this step exists requires a brief introduction to how ORE Studio organises data.

Tenants

A *tenant* is a fully isolated organisational space within an ORE Studio deployment. Each tenant has its own users, its own reference data (currencies, counterparties, books), and its own analytics results. Data belonging to one tenant is completely invisible to another tenant; there is no cross-tenant data leakage by design.

A typical deployment has one tenant per organisation. If you are running ORE Studio for your own firm, you will create one tenant representing your organisation. A managed-service provider running ORE Studio on behalf of multiple clients might create one tenant per client.

ORE Studio supports four tenant types, each with different operational controls:

- **System** — platform administration; one per deployment; platform-managed.
- **Production** — real customer organisations with live data; strict controls.
- **Evaluation** — demos, UAT, training, and exploratory testing; relaxed controls.
- **Automation** — programmatic test infrastructure; not for human use; no controls.

The **system tenant** is special — it is created automatically during bootstrapping and cannot be deleted. It is the home of the platform administrator accounts. There is exactly one system tenant per deployment. All other tenants are created by platform administrators working from the system tenant.

Production tenants are for live operations. They enforce strict controls including four-eyes authorisation for sensitive operations and KYC-gated counterparty onboarding.

Evaluation tenants provide a realistic but relaxed environment for demonstrations, user acceptance testing, and training. Bulk data import from external sources (such as the GLEIF/LEI registry) is available in evaluation tenants but not in production tenants.

Parties

Within a tenant, a *party* is a business unit — a legal entity, a branch, a trading desk, or any other organisational subdivision. Parties form a hierarchy: a parent party can see all data belonging to its children and grandchildren; a child party sees only its own data and that of its own descendants.

Every tenant has exactly one **system party**, created automatically when the tenant is provisioned. The system party is the administrative home for tenant administrator accounts. It is not a business entity and should not be used for trading activity.

Business parties — the entities that actually own trades, counterparties, and analytics results — are **operational parties**. You create these after the tenant is provisioned, using the Party Provisioner.

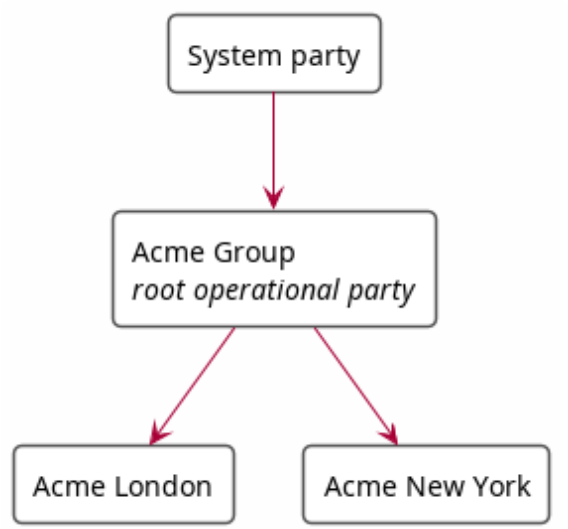

Figure 32: A simple party hierarchy: the system party at the top, the root operational party (Acme Group) below it, and two child operational parties (Acme London, Acme New York). Arrows point from parent to child.

A user logs in to a specific party within a tenant. Their data visibility is determined by their position in the hierarchy: a user logged in at "Acme Group" sees data from both "Acme London" and "Acme New York"; a user logged in at "Acme London" sees only their own data.

The bootstrapping sequence

A freshly installed ORE Studio backend goes through a fixed sequence before it is ready for normal use:

1. **System bootstrap** — the first platform administrator account is created and the system tenant is initialised. Until this step completes, the backend runs in *bootstrap mode* and accepts no normal user logins.

2. **Tenant provisioning** — one or more tenants are created by the platform administrator.
3. **Party provisioning** — within each tenant, the operational party hierarchy is established by the tenant administrator.

Each step has a corresponding wizard in the ORE Studio UI. The following sections walk through each one.

The System Provisioner wizard

The first time you connect to a backend that has never been initialised, ORE Studio detects *bootstrap mode* and launches the **New System Provisioner** automatically. This one-time wizard creates the platform administrator account, chooses a single- or multi-tenant layout, and provisions the first tenant together with its own administrator.

Welcome

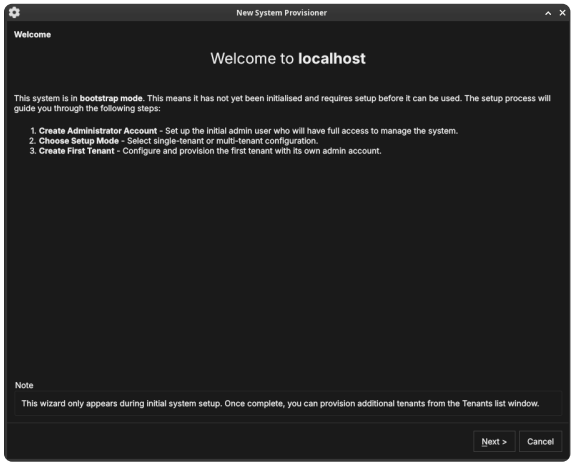

Figure 33: The System Provisioner welcome page, shown when the backend is in bootstrap mode. It lists the three setup steps.

The welcome page confirms the system is in bootstrap mode and outlines the steps: create the administrator account, choose the setup mode, and create the first tenant. Click **Next**.

Create the administrator account

Enter a **username**, **email**, and **password** (with confirmation) for the platform administrator. Keep these credentials safe — this account has unrestricted access to the entire deployment. Click **Create & Continue**.

Choose the setup mode

Choose how the deployment is organised:

- **Single-Tenant** — best for evaluation, development, or a single organisation. Creates a default tenant with sensible defaults; you can add more tenants later.

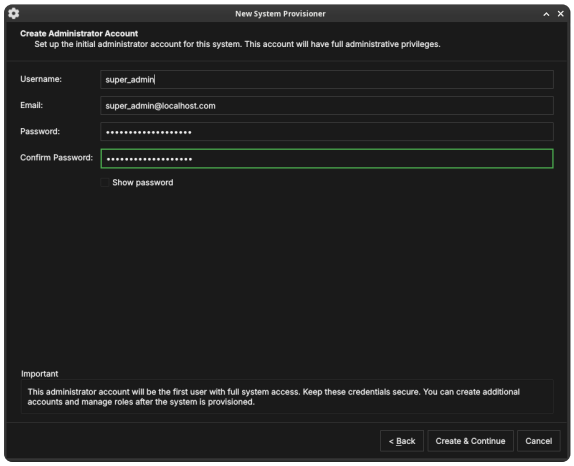

Figure 34: Creating the platform administrator account — the first account, with full administrative privileges over the whole deployment.

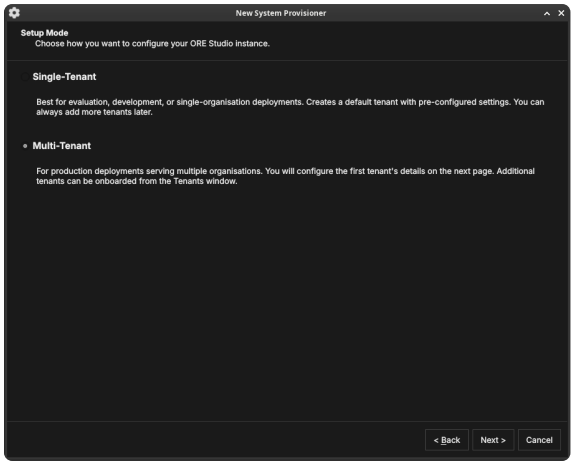

Figure 35: The Setup Mode page. *Single-Tenant* creates a default tenant with pre-configured settings; *Multi-Tenant* lets you configure the first tenant in detail and onboard more later.

- **Multi-Tenant** — for production deployments serving several organisations. You configure the first tenant's details on the next page and onboard additional tenants from the Tenants window.

Click **Next**.

Tenant details

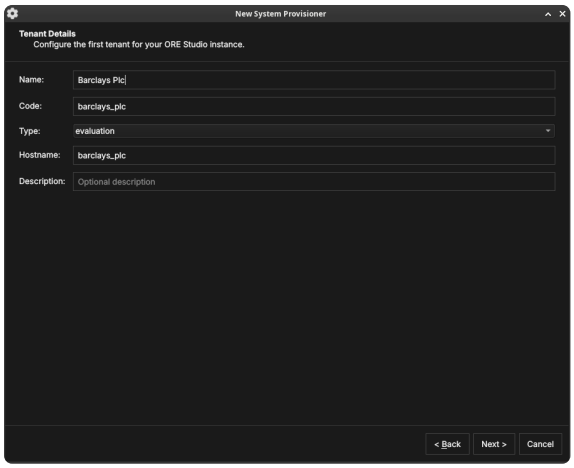

Figure 36: The first tenant's details: a display name, a short code, a tenant type, and a hostname.

Configure the first tenant:

- **Name** — the organisation's display name (e.g. "Barclays Plc").
- **Code** — a short machine code used internally (e.g. `barclays_plc`).
- **Type** — the tenant type (see Tenants above): *System*, *Production*, *Evaluation*, or *Automation*.
- **Hostname** — the host identifier for the tenant.

Click **Next**.

Tenant administrator account

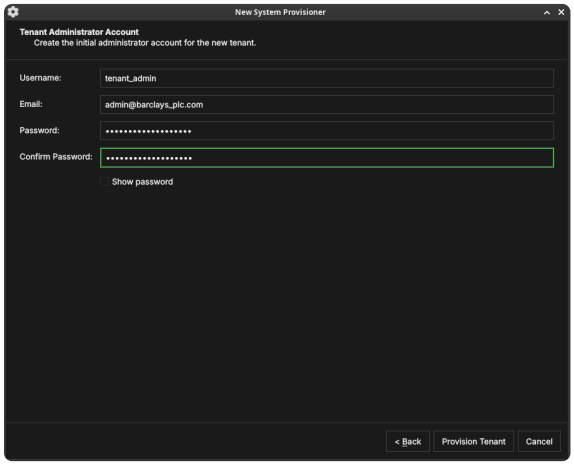

Figure 37: The tenant administrator account. This account administers the new tenant and is independent of the platform administrator.

Create the initial administrator for the new tenant — a **username**, **email**, and **password**. This account is given the `TenantAdmin` role: full control within the tenant, but no cross-tenant access. Click **Provision Tenant**.

Provisioning

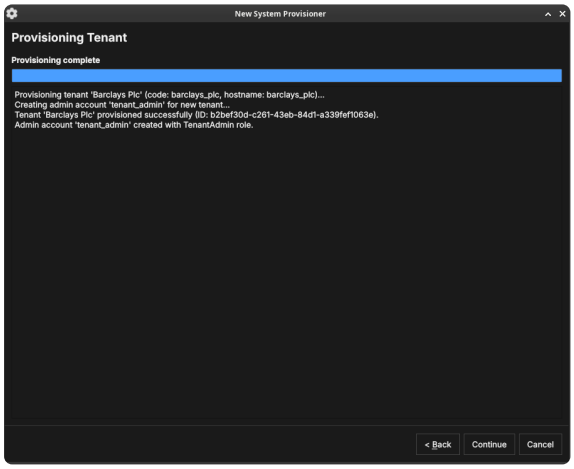

Figure 38: The provisioning step. ORE Studio creates the tenant and its administrator, logging progress as it goes.

ORE Studio provisions the tenant and creates its administrator account, logging each step. When it finishes, click **Continue**.

Setup complete

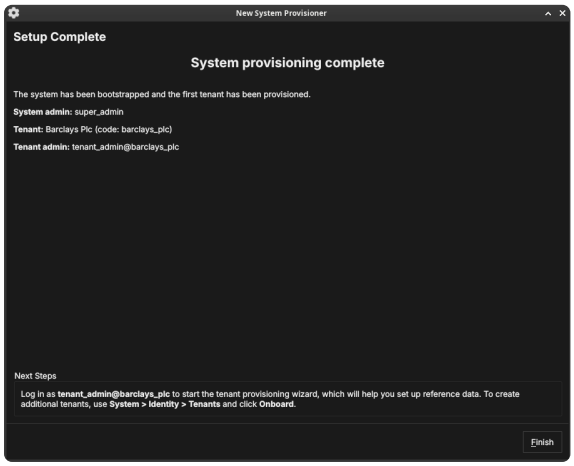

Figure 39: The System Provisioner summary: the platform admin, the first tenant, and the tenant admin that were created, with next-step guidance.

The final page summarises what was created and tells you the next step: log in as the tenant administrator to run the Tenant Provisioner. To create further tenants later, use *System* → *Identity* → *Tenants* and click **Onboard**. Click **Finish** — the backend leaves bootstrap mode and the login dialog appears.

The Tenant Provisioner wizard

The first time you log in as a tenant administrator, ORE Studio launches the **New Tenant Provisioner**. It seeds the tenant with reference data and, optionally, an initial party hierarchy. You can skip it with **Cancel** and set everything up by hand later from the Data Librarian and Parties windows.

Welcome

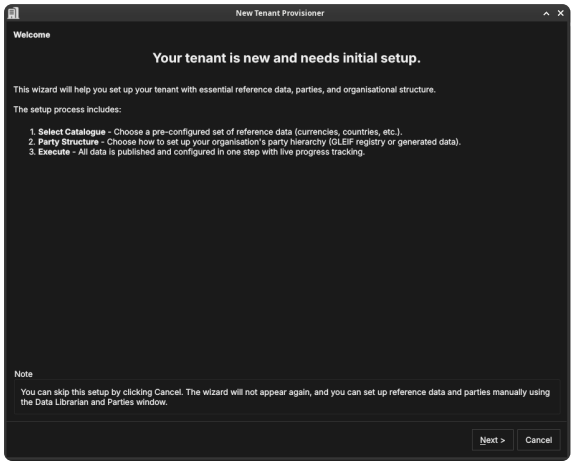

Figure 40: The Tenant Provisioner welcome page, listing its three steps: select a reference-data catalogue, choose a party structure, and execute.

The welcome page outlines the steps: select a catalogue, choose how to populate the party hierarchy, and execute. Click **Next**.

Select a catalogue

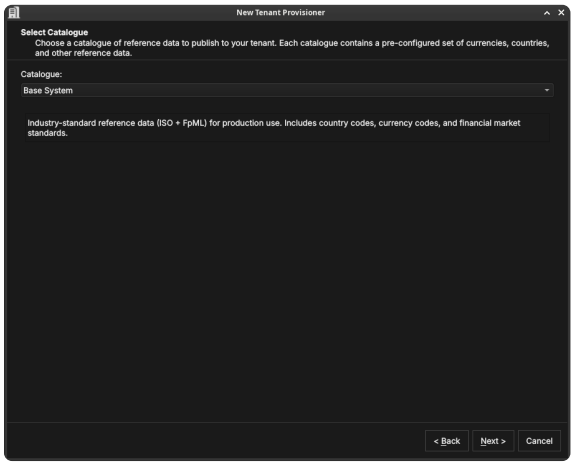

Figure 41: Selecting a reference-data catalogue — a pre-configured set of currencies, countries, and market standards.

A *catalogue* is a ready-made bundle of reference data. The *Base System* catalogue provides industry-standard ISO and FpML data — country codes, currency codes, and financial-market standards — suitable for production use. Choose a catalogue and click **Next**.

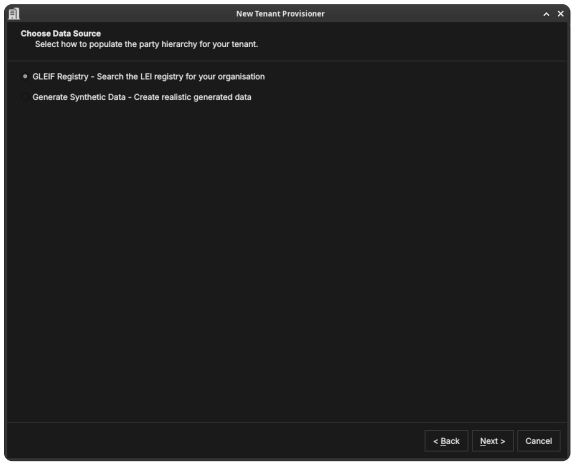

Figure 42: Choosing how to populate the party hierarchy: search the GLEIF LEI registry, or generate realistic synthetic data.

Choose a data source

Decide how the tenant's party hierarchy is seeded:

- **GLEIF Registry** — search the global LEI registry for your real organisation and import its structure.
- **Generate Synthetic Data** — create realistic generated parties, useful for evaluation and testing.

Click **Next**.

Party setup (optional)

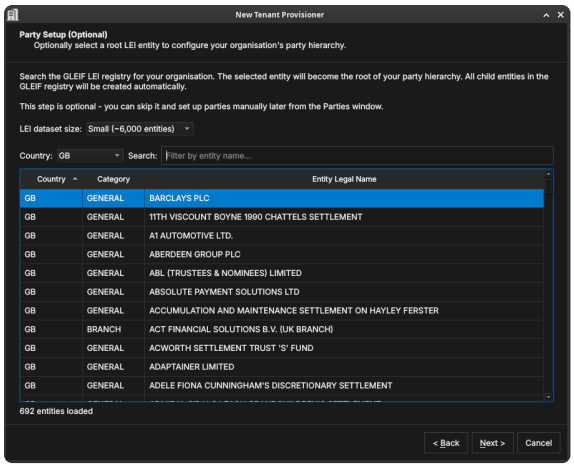

Figure 43: The optional Party Setup step. Search the GLEIF LEI registry, pick a root entity (here Barclays PLC), and its child entities are created automatically.

If you chose the GLEIF registry, search it for your organisation: pick an **LEI dataset size**, filter by **Country**, and search by entity name. The entity you select becomes the root of your party hierarchy, and its child entities in the registry are created automatically. This step is optional — skip it to set parties up manually later. Click **Next**.

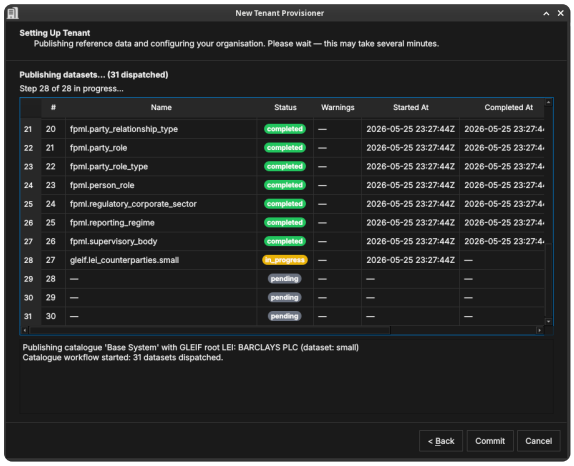

Figure 44: The publishing step. The catalogue and party data are published to the tenant in one operation, with live progress tracking.

Publishing

ORE Studio publishes the selected catalogue and party data to the tenant, tracking progress per dataset.

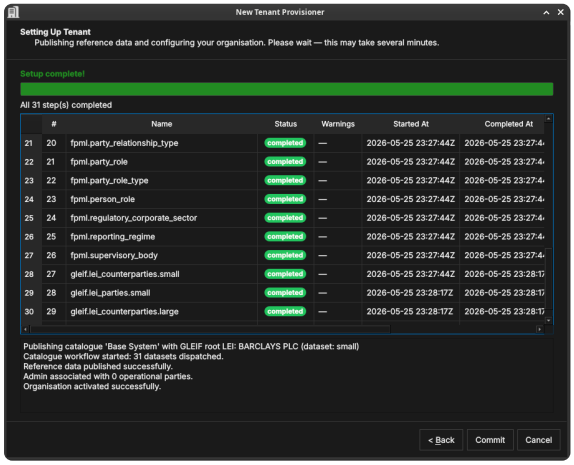

Figure 45: Publishing complete — all datasets have been published to the tenant.

Setup complete

The final page confirms what was created. Click **Finish**. The tenant now has its reference data and party hierarchy in place.

The Party Provisioner wizard

Where the Tenant Provisioner establishes the party *hierarchy*, the **Party Setup** wizard configures the operational structure *within* a party: it imports counterparties and creates a standard set of risk-report definitions. It runs for a party that needs initial setup, and can be skipped with **Cancel** (counterparties and reports can be configured later from the application menus).

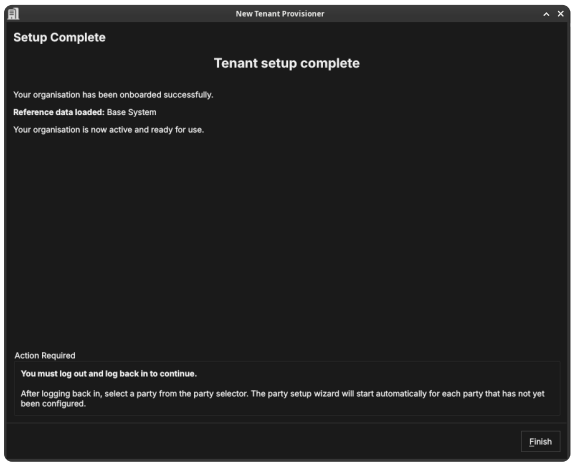

Figure 46: The Tenant Provisioner summary, confirming the reference data and parties that were set up.

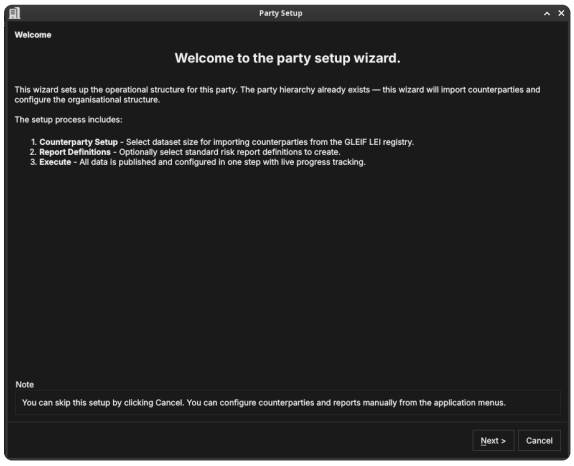

Figure 47: The Party Setup welcome page, listing its steps: counterparty import, report definitions, and execute.

Welcome

The welcome page explains that the party hierarchy already exists and that this wizard imports counterparties and configures the organisational structure. Click **Next**.

Counterparty import

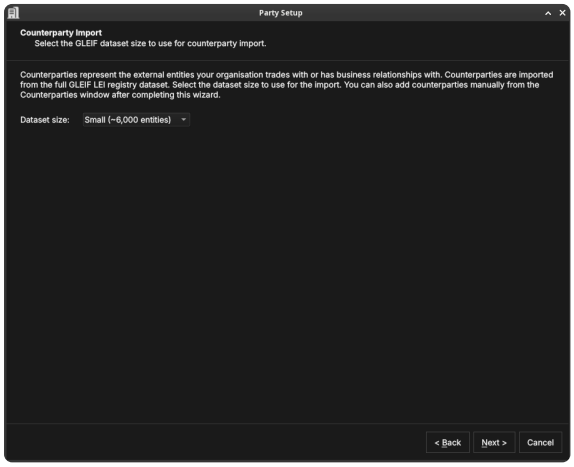

Figure 48: Choosing the GLEIF dataset size for importing counterparties — the external entities your organisation trades with.

Counterparties are the external entities your organisation trades with. They are imported from the GLEIF LEI registry; choose the **dataset size** to control how many are imported. You can add more later from the Counterparties window. Click **Next**.

Report definitions

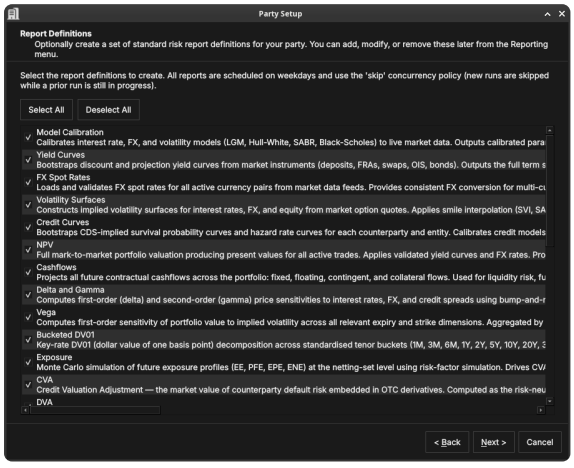

Figure 49: Selecting standard risk-report definitions to create — NPV, sensitivities, exposure, CVA, and more. All are scheduled on weekdays.

Optionally create a set of standard risk-report definitions — model calibration, yield curves, FX spot rates, volatility surfaces, credit curves, NPV, cashflows, Delta and Gamma, Vega, bucketed DV01, exposure, CVA, DVA, and others. Use **Select All** / **Deselect All**, or tick individual reports; they can be edited later from the Reporting menu. Click **Next**.

Execute

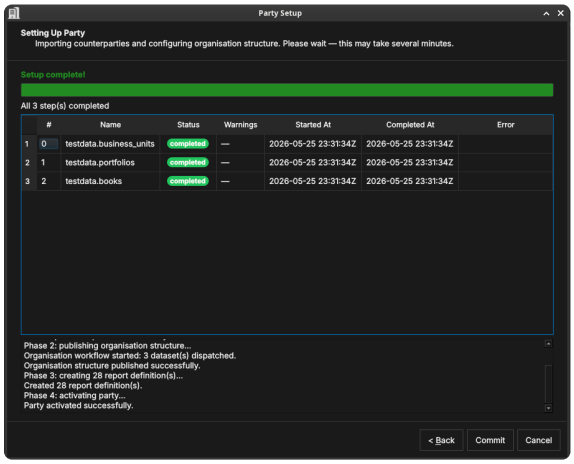

Figure 50: The execute step. Counterparties are imported and the organisation structure (business units, portfolios, books) and report definitions are created, with a live progress table and log.

ORE Studio runs the setup in phases — importing counterparties, publishing the organisation structure (business units, portfolios, books), creating the report definitions, and activating the party. When it completes, click **Commit**.

Setup complete

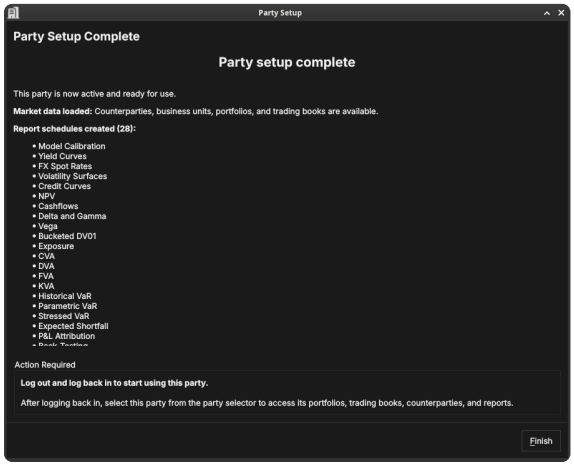

Figure 51: The Party Setup summary, confirming the counterparties, structure, and reports that were created.

The final page confirms what was set up. The party is now fully operational: it has counterparties, an organisational structure, and scheduled risk reports.

Choosing a party at login

Some accounts are associated with more than one party — for example a group administrator who can act for several entities. When you log in with such an account, ORE Studio asks which party context to use for the session.

Choose **System Party** for administrative work, or **Operational Party** to work within a specific business entity. Filter the list by booking **centre**, or type in the search box to narrow it.

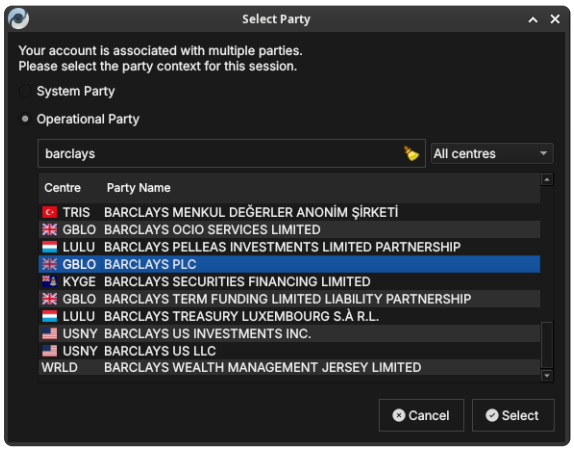

Figure 52: The Select Party dialog. Choose the System Party or an Operational Party; filter by booking centre or search by name.

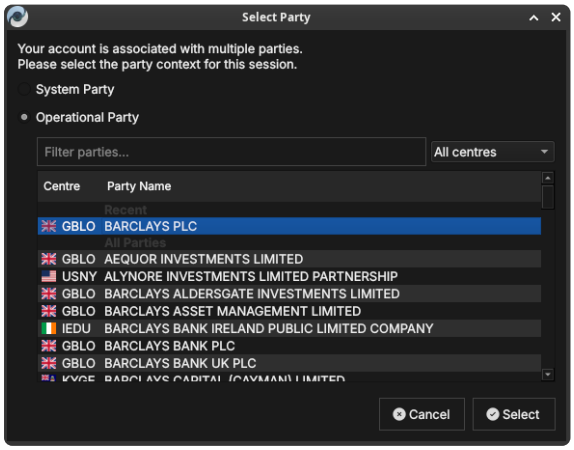

Figure 53: The Select Party dialog with a recently used party listed under *Recent* at the top for quick access.

Select a party and click **Select**. Your data visibility for the session is determined by the party you choose and its position in the hierarchy.